

DEPLOYMENT GUIDELINE 2 OF N • PLAIN-LANGUAGE WALKTHROUGH

Security — Explained

Your boss's team ran a security audit against the Core Health codebase — 02-security.md. This is section 2 of that audit, translated into plain language, with every finding checked against the code as it stands today (some were already partly fixed by earlier work this session — called out explicitly, not silently assumed).

SOURCE: 02-security.md P0 ITEMS: 10 + 7 sub-findings P1 ITEMS: 3

1 How to read this one

Same system as the architecture audit, with one important difference in what the findings are *about*.

PASS / FAIL / PARTIAL / CANNOT VERIFY / N/A

Same meanings as before — see the architecture guideline doc if this is the first one you're reading.

The important difference from the architecture audit

Most **CANNOT VERIFY** items in this document aren't "undocumented code" — they're questions about how your *team* operates (does everyone use MFA on GitHub? Is there an offboarding checklist?). Those can't be fixed by editing a file — only you and your boss can answer them. This document separates those clearly from the ones that are genuinely code fixes.

ALREADY RESOLVED BY EARLIER WORK

Two things this audit flags were already fixed earlier in this session, for unrelated reasons — confirmed against the current code before writing this, not assumed: (1) the dev-only localhost CORS origins are already restricted to non-production (ARCH-04's fix), and (2) the server's Dockerfile is already multi-stage, so the compiler toolchain already doesn't ship in the production image (done while fixing ARCH-06's file storage). Both are called out again below in their specific findings, marked done.

ONE NEW CONFLICT FOUND WHILE CHECKING THIS

ARCH-06 (the earlier S3 migration work) added a route to serve uploaded files — GET /uploads/:subdir/:filename — that fixed the "files disappear on redeploy" problem but is

NOT AUTHENTICATED

. This audit's SEC-28 flags exactly that: uploaded prescriptions and lab reports are reachable by anyone who can guess or discover a filename, no login required. Confirmed still true by re-reading server.ts just now. This needs fixing — details in §3.

2 The urgent one, already handled

SEC-31 — the audit flagged that 7 real-looking uploaded files (5 prescription images, 1 more, 1 lab report PDF) were committed into git back in June. Confirmed: they were. You confirmed these are test/sample images, not real patient data — so this was git hygiene, not a security incident. Already fixed: the files are untracked from git going forward (still present in older history — a full history rewrite would remove that too, but that's a separate, more disruptive decision, not done without asking first since it affects shared history).

3 The P0 items — what's wrong, why it matters, how to fix it

SEC-07 / SEC-21 / SEC-29 — Default admin password, printed and documented

P0

PARTIAL

The seed script hardcodes `admin123` as the admin password, prints "Admin credentials: `admin@gmail.com / admin123`" to the console on every run, and the same password is written in two README-style documents and baked into the schema dump as a real password hash.

WHY IT MATTERS

Anyone who reads the repo — or the server's startup logs — has the admin password. This is the single most impactful finding in the whole document: full admin access requires zero effort to obtain.

HOW TO FIX IT

- 1 Read the seed password from an environment variable instead of hardcoding it; if that variable isn't set, don't seed an admin at all rather than falling back to a known value.
- 2 Delete the `console.log` line that prints it.
- 3 Remove the password from both README-style documents.
- 4 Make sure no account with this password exists in anything resembling production — if one does, rotate it.

SEC-14 — No MFA, no IP restriction for admin

P0

FAIL

The admin area is just a normal part of the app — same login, same domain, same network path as a patient's. Access control is entirely "does this JWT say `role = admin`." No extra verification, no network restriction. Admins can also impersonate any other user.

WHY IT MATTERS

If one admin password leaks (phishing, reused password, etc.) the attacker has full access to every patient's medical data immediately — there's no second barrier at all.

HOW TO FIX IT — TWO TIERS

- 1 **Quick, partial mitigation:** shorten how long an admin's login token stays valid (today it's the same 7 days as everyone else) — this is a small code change.
- 2 **The real fix, bigger:** require MFA (a one-time code from an app, typically) for admin logins specifically. This is a feature to design, not a one-line fix — needs a decision on method (authenticator app vs. email code) before building. Flagged for your input, not started yet.
- 3 **Further hardening (optional, bigger):** serve admin routes on a separate subdomain restricted by IP allow-list or VPN — an infrastructure change, fits naturally into the AWS migration later rather than Railway today.

SEC-17 — Access control & authentication (7 separate gaps)

P0

PARTIAL

This one audit ID covers a full OWASP-style review. Injection and XSS both pass cleanly. Access control and authentication do not — seven distinct problems, each real:

1 — UPLOADED MEDICAL FILES ARE PUBLIC

Same issue as SEC-28 below — covered once there, not twice here.

2 — SELF-REGISTERED DOCTORS/PHARMACISTS/LABS ARE ACTIVE IMMEDIATELY

Anyone can sign up choosing "doctor" as their role, and the account is fully active the instant they submit the form — no license check, no email verification, no human approval. That "doctor" can immediately search for patients by name or email.

WHY IT MATTERS

There's currently nothing stopping a stranger from registering as a doctor and browsing patient names. This is a real, exploitable gap today, not a theoretical one.

3 — DEACTIVATED USERS KEEP WORKING UNTIL THEIR TOKEN EXPIRES

The protect middleware (the thing every authenticated route runs through) only checks that the JWT signature is valid — it never re-checks the database to see if that user still exists or is still active. If an account gets suspended, it keeps working for up to 7 more days.

WHY IT MATTERS

"I deactivated that account" currently doesn't mean what everyone assumes it means.

4 — MEDICINES/SUPPLIERS/INVENTORY ARE SHARED ACROSS EVERY PHARMACY

These tables live in the shared `public` schema with no organization boundary — any logged-in pharmacist can edit the price and stock of *any* pharmacy's medicines, not just their own. Any patient can read all of it too.

WHY IT MATTERS

This breaks the core multi-tenant promise for this one slice of data — "my pharmacy's inventory" currently means "everyone's inventory."

5 — A PUBLIC ENDPOINT LEAKS STAFF NAMES, EMAILS, AND LICENSE NUMBERS

The org-registration page's "search for an existing owner" box is unauthenticated by design (registration happens before any login exists) — but it searches *all* active doctors/pharmacists/labs system-wide and returns name, email, and license number to anyone who types 2+ characters, logged in or not.

6 — PATIENT HISTORY HAS NO CONSENT CHECK IN APPLICATION CODE

A doctor can fetch any patient's name and email by ID with no check that they actually have a relationship with that patient — the clinical details are protected by database-level row security, but that protection does nothing if the app connects to the database as a superuser (which is common, and a separate database-level finding in the next guideline document).

7 — IMPERSONATION IS ONLY LOGGED WHEN IT STARTS

When an admin impersonates a user, that event is recorded — but nothing they *do* while impersonating is. If that access were misused, there'd be no trail of what actually happened.

PLUS, AUTHENTICATION ITSELF

Minimum password length is 6 characters. No limit on login attempts (ties to SEC-22 below). Tokens last 7 days with no way to revoke one early — if a token leaks, there's no "kill switch" short of waiting out the week.

SEC-18 — No dependency vulnerability scan

P0

FAIL

Nothing in CI checks whether any of this project's npm packages have known security vulnerabilities.

HOW TO FIX IT

Add `npm audit --audit-level=critical` (or an equivalent GitHub Action) for both server and client, set to fail the build on a hit.

SEC-19 — Docker image hardening

P0

FAIL

Four sub-issues, listed as they stand today (not all are still true):

ALREADY FIXED

"Single-stage image, compiler toolchain ships to production" — no longer true. The server Dockerfile was rebuilt multi-stage during the earlier AWS/ARCH-06 work specifically to keep g++/python3/make out of the final image.

- 1 **Still true:** both containers run as root — no USER line in either Dockerfile.
- 2 **Still true:** the base image tag `node:20-slim` floats — it'll silently pick up a different image over time instead of a pinned, known-good one.
- 3 **Still true:** nothing scans the built image for vulnerabilities (a Trivy step in CI, typically).

SEC-20 — No security headers

P0

PARTIAL

ALREADY FIXED

"localhost origins allowed in production" — no longer true, fixed by ARCH-04 earlier this session (server/config/corsOrigins.ts).

Still true: no security headers at all on either the API or the static frontend — no HSTS (forces HTTPS), no nosniff (stops MIME-sniffing attacks), no frame-blocking (clickjacking protection), no Content-Security-Policy.

HOW TO FIX IT

- 1 Add the `helmet` package to the Express app with a real Content-Security-Policy, not just the defaults.
- 2 For the static client: add a `serve.json` config (the `serve` package already in use supports custom headers) or set them at whatever eventually sits in front of it (CloudFront, once that's live).

SEC-21 — Error handling leaks detail, no fail-fast

P0

PARTIAL

A 500 error currently returns the raw error message — including actual database error text — straight to the client, always, regardless of environment. Stack traces are hidden only when `NODE_ENV=production` happens to be set; if it's missing entirely, nothing hides them.

HOW TO FIX IT

- 1 Return a generic "something went wrong" message for any 500, and log the real detail server-side only.
- 2 Refuse to start the server at all if `NODE_ENV` isn't set to something recognized — don't silently assume development.

SEC-22 — No rate limiting anywhere

P0

FAIL

Login, registration, and public organization registration can all be hit as fast as a script can send requests — nothing slows down repeated attempts.

WHY IT MATTERS

This is what makes password-guessing attacks practical. Without a limit, trying thousands of passwords against one account costs an attacker nothing but time.

HOW TO FIX IT

- 1 Add `express-rate-limit` with strict limits on `/api/auth/login`, `/api/auth/register`, `/api/organizations/register`, and the two public search endpoints.
- 2 Set `app.set('trust proxy', 1)` so the limiter counts the real client IP, not whatever proxy sits in front (Railway today, the ALB later).

SEC-28 — Uploaded medical files are public

P0

FAIL

Confirmed still true, including through the route added during the earlier S3 migration work: `GET /uploads/:subdir/:filename` has no login check at all. File names also follow a guessable pattern (`rx_{userId}_{timestamp}`), so someone doesn't even need a link — they can construct plausible ones.

WHY IT MATTERS

Prescriptions and lab reports are exactly the kind of data a healthcare app exists to protect. Right now, anyone with a URL pattern and a little patience can potentially view another patient's medical documents with zero authentication.

HOW TO FIX IT

- 1 Remove the unauthenticated static-style route entirely.
- 2 Serve every file through an endpoint that requires login *and* checks the requester actually owns (or has a legitimate clinical relationship to) that file — the pattern already exists correctly in `patientReportController.ts`'s `serveFile/serveFileForDoctor` functions; the other file types need the same treatment.
- 3 Use random, unguessable file names (see SEC-24) instead of the predictable `rx_{userId}_{timestamp}` pattern — defense in depth, in case an auth check is ever missed somewhere.

SEC-32 — Node.js 20 reached end of life

P0

PARTIAL

Node.js 20 stopped receiving security updates in April 2026. Both Dockerfiles and CI still target it.

HOW TO FIX IT

Move to the current Node.js LTS (22) in both Dockerfiles and the CI workflow.

4 P1 — should fix soon, not blocking

SEC-15 — No rate limiting, no WAF/CDN

P1

FAIL

Same underlying gap as SEC-22 (no rate limiting) plus nothing in front of the app for broader protection. The rate-limiting half gets fixed by SEC-22's work directly. The CDN/WAF half naturally arrives once the AWS edge module (CloudFront) is live.

SEC-24 — Upload validation is shallow

P1

PARTIAL

File size limits exist and work. But the file-type check only looks at the extension in the filename (easy to fake), and filenames are predictable rather than random.

HOW TO FIX IT

Check the real file type from its actual content (a package like `file-type` reads the file's binary signature, not its name) and generate names with `crypto.randomUUID()` instead of the current pattern.

SEC-25 — No static security analysis; TypeScript strict mode is off

P1

FAIL

ESLint runs in CI today but is configured to never fail the build even when it finds something. There's no CodeQL/Semgrep-style security-focused scanner at all.

HOW TO FIX IT

- 1 Remove the `continue-on-error` flag so ESLint failures actually block a merge.
- 2 Add CodeQL (free, built into GitHub) or Semgrep on pull requests.
- 3 Consider turning on TypeScript's strict mode — bigger, separate effort, likely surfaces a batch of new type errors to work through, not a quick toggle.

5 Not code — needs you or your boss, not a fix from me

These aren't gaps in the codebase — they're questions about how your team and cloud accounts are operated. I can't answer or fix these by editing files.

ID	WHAT IT'S ACTUALLY ASKING	WHO CAN ACTUALLY ANSWER IT
SEC-01	Is MFA enabled on your cloud account, GitHub, domain registrar, DNS provider, and email?	You/your boss — account settings, not code
SEC-02	Does every team member have their own named login, or is anything shared?	Same
SEC-03	Does each person/service have only the access they actually need?	Same
SEC-05 / 06	Is there a real process for removing access when someone leaves, and reviewing who has access periodically?	Same
SEC-08 / 09 / 10	Where are secrets actually stored, are they different per environment, is there a rotation plan?	Partly code (already passes: secrets come from env vars), partly process
SEC-11 / 12 / 13	Is the database in a private network, is there a firewall, are certificates managed properly?	Infrastructure — resolved automatically once the AWS migration (already designed) lands
SEC-16	Is there DDoS protection in front of the app?	Infrastructure — CloudFront (already planned) includes AWS Shield Standard by default
SEC-23	PASS — already confirmed: bcrypt cost 10, tokens expire.	No action
SEC-26	Has a real penetration test been run?	You — needs to be commissioned, not something I can do
SEC-27	Is data encrypted at rest?	Infrastructure — RDS encryption is already in the Terraform (storage_encrypted = true), S3 buckets use SSE-S3 by default
SEC-30	Is there a written inventory of what personal data is stored and why?	You/your boss — a compliance document, not code

SEC-33 / 34	Is there a patching schedule, and cloud-side audit logging?	Process + AWS CloudTrail (infrastructure, not yet built)
-------------	---	--

6 Suggested order of work

Grouped by what's actually involved, not just severity — some POs are a few lines, others are real feature decisions.

Tier 1 — pure code, no design decisions needed, do these first

- 1 SEC-07/21/29 — remove the hardcoded admin password and the console print.
- 2 SEC-21 — generic 500 messages, fail-fast if `NODE_ENV` isn't set.
- 3 SEC-22 / SEC-15 — rate limiting on auth + registration endpoints.
- 4 SEC-20 — add `helmet` with a real CSP.
- 5 SEC-18 — `npm audit` in CI.
- 6 SEC-25 — remove `continue-on-error`, add `CodeQL`.
- 7 SEC-19 — add `USER node`, pin the base image, add a Trivy scan.
- 8 SEC-32 — bump to Node 22 in both Dockerfiles and CI.
- 9 SEC-17 #3 — make `protect` check the user still exists and is active.
- 10 SEC-17 #5 — tighten the public owner-search endpoint's exposed fields.
- 11 SEC-17 #6 — add an explicit consent check in `getPatientHistory`.
- 12 SEC-24 — real content-type checking + random filenames.
- 13 SEC-28 — remove the unauthenticated `/uploads` route, serve every file type through an authenticated, ownership-checked endpoint.

Tier 2 — real feature work, needs your input on approach before building

- 1 SEC-14 — MFA for admin accounts (method TBD — authenticator app vs. email code).
- 2 SEC-17 #2 — an approval step for doctor/pharmacist/lab self-registration instead of instant activation.
- 3 SEC-17 #4 — tenant-scope medicines/suppliers/inventory (currently shared across every pharmacy) — likely needs a schema change, bigger effort.
- 4 SEC-17 (auth) — token revocation / shorter-lived tokens with refresh, instead of one 7-day token with no kill switch.
- 5 SEC-17 #7 — log actions taken during impersonation, not just the start.

Tier 3 — not code at all

Everything in §5 above — these need you and your boss to confirm or action directly.

Core Health — plain-language walkthrough of 02-security.md. Checked against the current code before writing, not assumed from the source document alone.